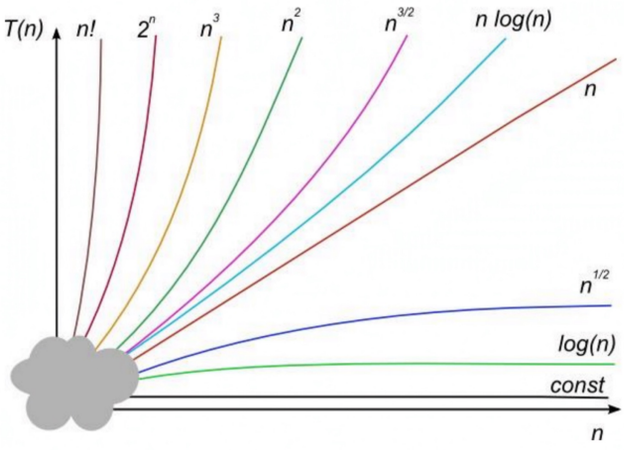
2022.02.14.

# 2. Gyakorlat

## Nevezetes nagyságrendek

- példák különféle műveletigényű algoritmusokra

| | |
|---|---|
| • Verem vagy sor bármely művelete | $\Theta(1)$ |
| • Logaritmikus (=bináris) keresés | $\Theta(\log n)$ |
| • Prímszámteszt ($n^{1/2}$-ig) | $\Theta\ (n^{1/2})$ |
| • Lineáris keresés | $\Theta(n)$ |
| • Kupacrendezés | $\Theta(n\ \log n)$ |
| • Shell rendezés | $\Theta(n^{3/2})$ |
| • Buborékrendezés | $\Theta(n^2)$ |
| • Mátrixszorzás | $\Theta(n^3)$ |
| • Hanoi tornyai | $\Theta(2^n)$ |
| • Utazó ügynök probléma | $\Theta(n!)$ |



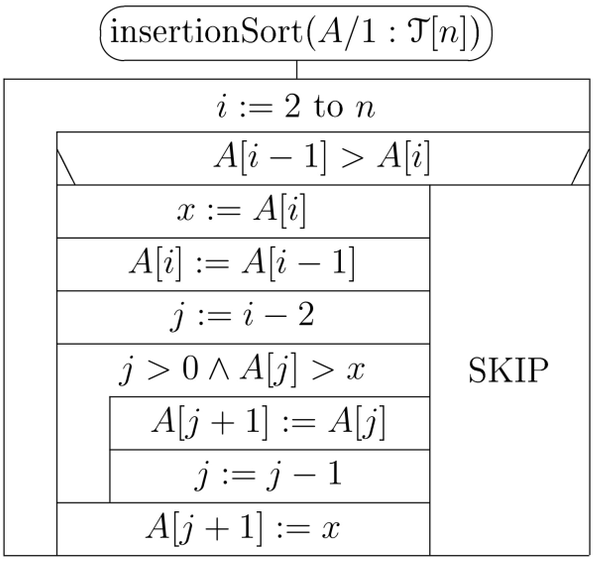

**1.6. ábra.** A gyakran előforduló nagyságrendek grafikonja

## Beszúró rendezés

- Kezdetben: rendezett résztömb (az első elem) + rendezetlen résztömb
- Egy lépés: a rendezetlen résztömbből elveszünk egy elemet, és beszúrjuk a rendezett résztömbbeli helyére
- *n-1* beszúrás után rendezett a tömb
- tömbökre és láncolt listákra is alkalmazható

A[1 : k]

A[1 : k+1]




• rendezett rész a tömb elején, a rendezetlen rész utána (kezdetben csak a tömb első eleme rendezett)

• a beszúrandó elem *x*-ben

• külső ciklus: beszúrásonként lép egyet, addig megy, amíg nem kerül minden elem a helyére

• belső ciklus: a beszúrás közbeni másolások, addig megy, amíg meg nem találtuk az aktuálisan beszúrandó elem helyét

- a külső ciklus mindig n-1 alkalommal fut le (ennyi elemet szúrunk be)
- belső ciklus műveletigénye:
  - legjobb esetben: egy összehasonlítás, nincs másolás

- mÖ(n) = 1 + (n - 1) = Θ(*n*)
- mM(n) = 2 (n - 1) = Θ(*n*)

- legrosszabb esetben: annyi összehasonlítás és másolás, ahány rendezett elem

$$MT_{IS}(n) = 1 + (n-1) + \sum_{i=2}^{n}(i-2) = n + \sum_{j=0}^{n-2} j = n + \frac{(n-1)*(n-2)}{2}$$

$$MT_{IS}(n) = \frac{1}{2}n^2 - \frac{1}{2}n + 1 \simeq \Theta(n^2)$$

$$AT_{IS}(n) \approx 1 + (n-1) + \sum_{i=2}^{n}\left(\frac{i-2}{2}\right) = n + \frac{1}{2} * \sum_{j=0}^{n-2} j =$$

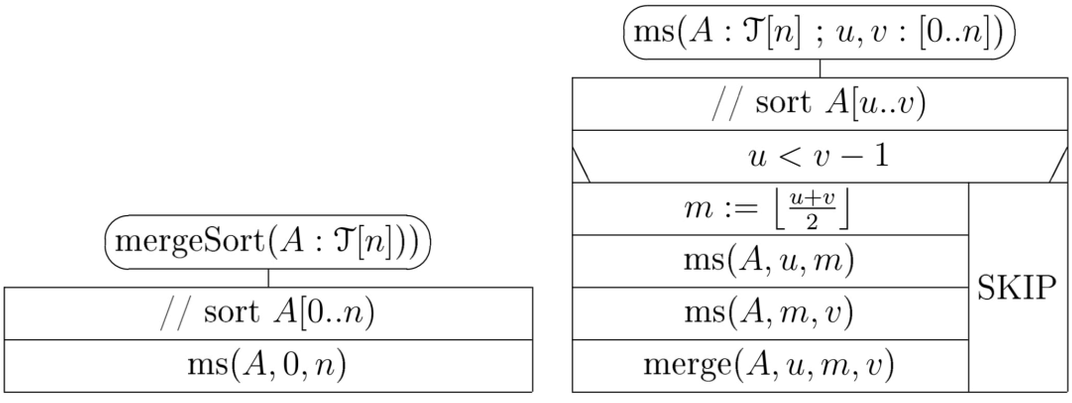
$$= n + \frac{1}{2} * \frac{(n-1)*(n-2)}{2} = \frac{1}{4}n^2 + \frac{1}{4}n + \frac{1}{2} \simeq \Theta(n^2)$$

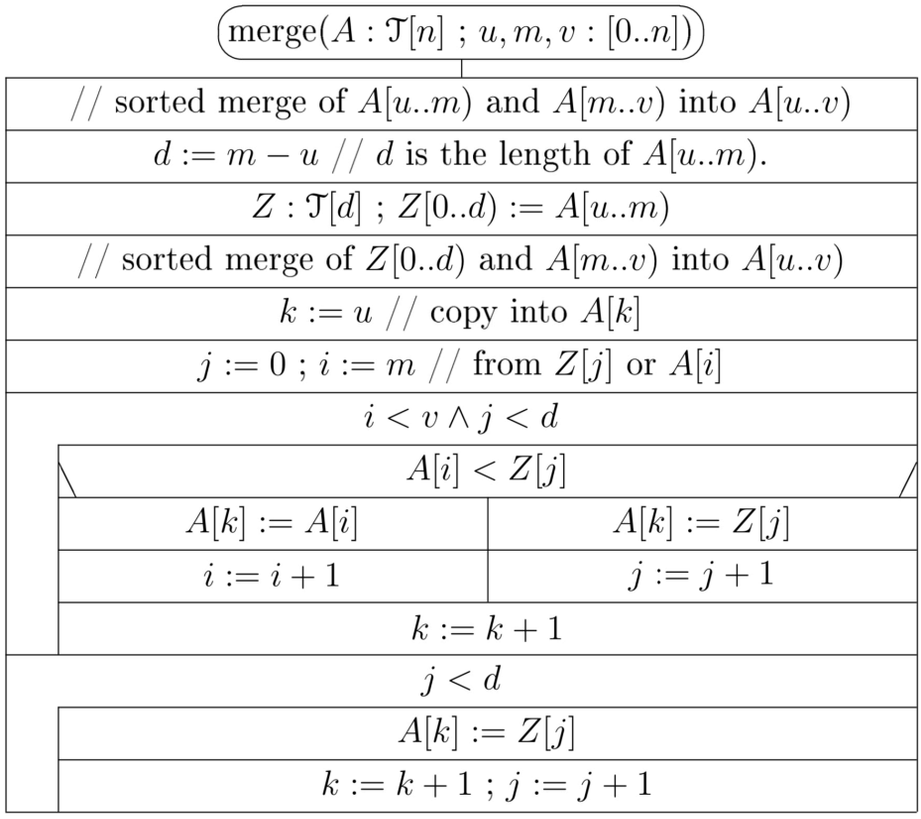
### Rendezés stabilitása

Egy rendező eljárást stabilnak nevezünk, ha nem változtatja meg az egyenlő kulcsú elemek egymáshoz viszonyított sorrendjét.

## Összefésülő rendezés

- Rekurzív megközelítés:
  - “oszd meg és uralkodj”
    - a rekurzió minden szintjén a következő három lépés:
      1. A feladat több részfeladatra osztása, amik hasonlóak az eredeti feladathoz, de kisebb a méretük.
      2. Rekurzív módon megoldja a részfeladtokat.
      3. A részfeladatok meoldásait kombinálva megadja az eredeti feladat megoldását.





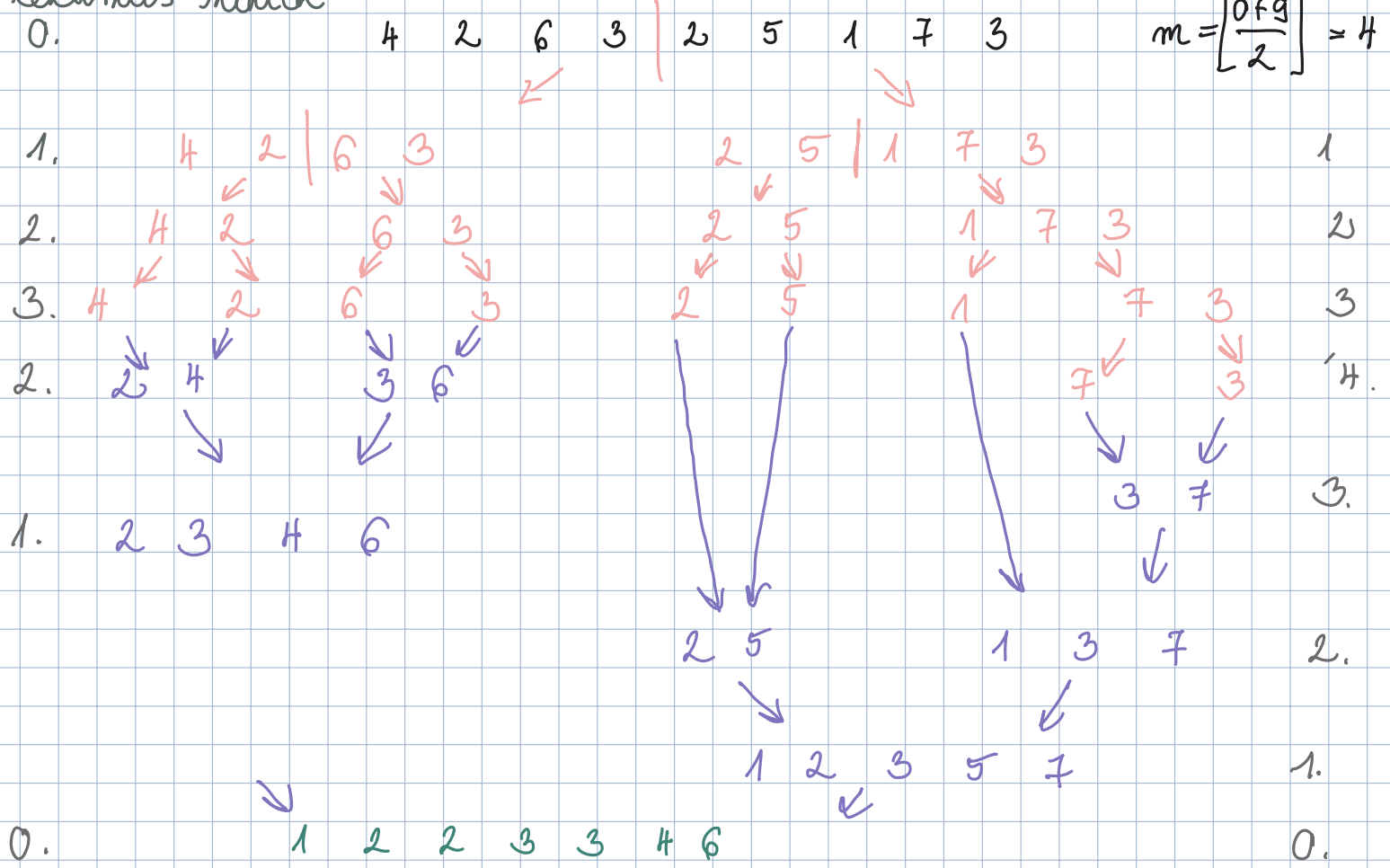

- **Műveletigény**

$$MT_{MS}(n), mT_{MS}(n) \in \Theta(n \log n)$$

- a merge(A,u,m,v) eljárás műveletigényének meghatározásához vezessük be az l=v-u jelölést

Rendezzük a ⟨4, 2, 6, 3, 2, 5, 1, 7, 3⟩

Rekurziós szintek



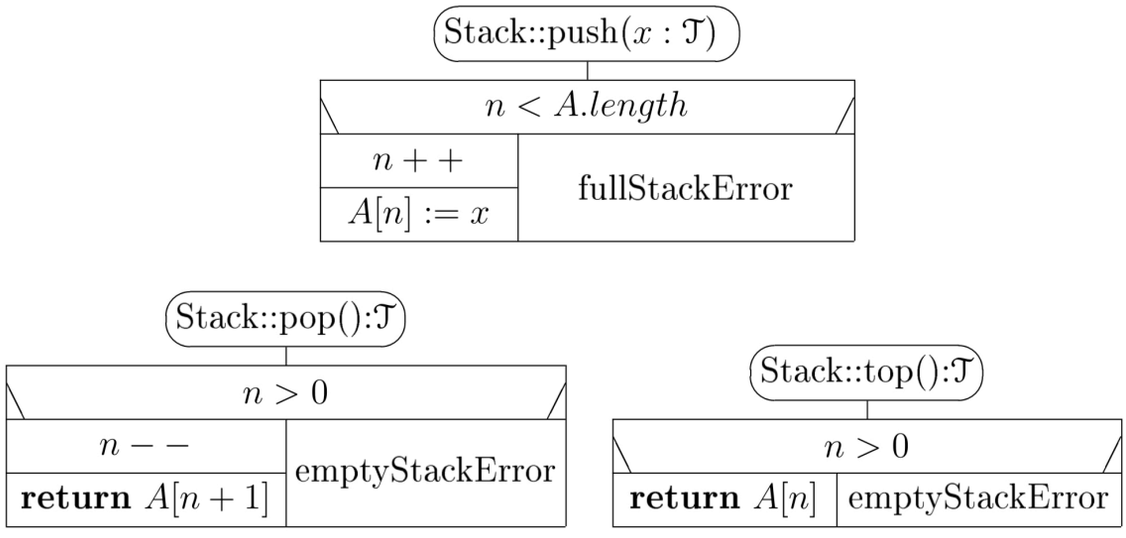
## Verem adattípus




| **Stack** |
|---|
| - $A/1 : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $A.length$ is the max. size of the stack<br>- $n : \mathbb{N}$ // $n \in 0..A.length$ is the actual size of the stack |
| + Stack($m : \mathbb{N}$) $\{A := \textbf{new } \mathcal{T}[m]$ ; $n := 0\}$ // create an empty stack<br>+ $\sim$ Stack() { **delete** $A$ }<br>+ push($x : \mathcal{T}$) // push $x$ onto the top of the stack<br>+ pop() : $\mathcal{T}$ // remove and return the top element of the stack<br>+ top() : $\mathcal{T}$ // return the top element of the stack<br>+ isFull() : $\mathbb{B}$ $\{$**return** $n = A.length\}$<br>+ isEmpty() : $\mathbb{B}$ $\{$**return** $n = 0\}$<br>+ setEmpty() $\{n := 0\}$ // reinitialize the stack |

- LIFO (last in first out) adattároló
  - mindig csak az utoljára beletett, és még benne levő adat érhető el, és törölhető
- dinamikus tömbbel reprrezentáljuk: A/1 : T[ ]

### A verem műveletei

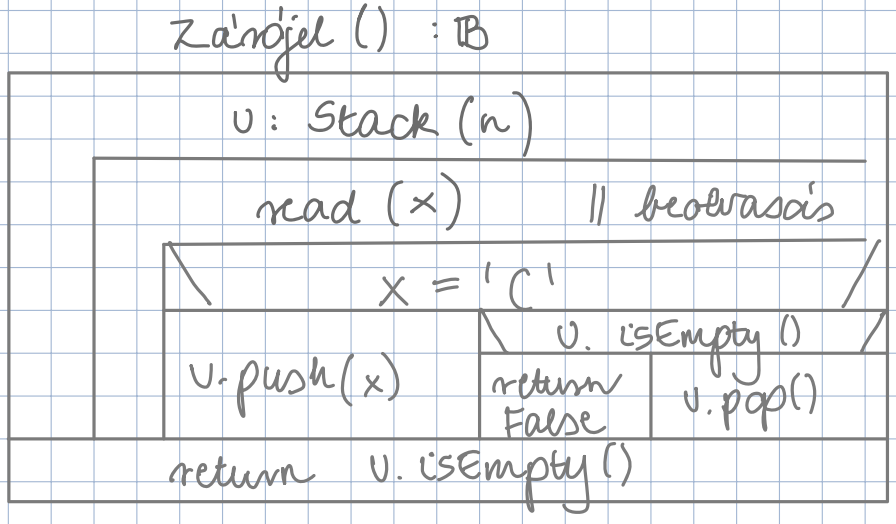


# Veremmel megoldható feladatok

## 1. Helyes zárójelezés

input: max n hosszú karaktersorozat

1. minden ( van ) párja
2. sorrend




## 2. Neptunfordító

input: András, Béla, Csaba → Csaba, Béla, András